

Лифт 5: Без права на остановку

Всю логику вы реализуете с помощью создания API на ASP.NET и создания БД на PostgreSQL. Важные особенности:

- БД создаётся с помощью метода CodeFirst (то есть сначала появляются классы в проекте и на их основе выполняются миграции, то есть контекст вы прописываете самостоятельно)
- Сам проект мы упаковываем в докер, он должен работать из контейнера
- Не забудьте использовать логи

Основное задание «Вертикальный город»

Вы устроились в компанию, которая строит «умные» бизнес-центры. Ваша задача — создать ядро системы управления лифтами. Система должна не просто двигать кабины по шахтам, но и собирать детальную статистику для диспетчеров и бухгалтерии (например, для расчета износа механизмов). Вам предстоит спроектировать REST API и базу данных.

База данных

Обязательные таблицы:

- **buildings** — здания (адрес, количество этажей).
- **elevators** — конкретные лифтовые установки (серийный номер, модель, дата производства, к какому зданию привязаны, мин/макс этаж, скорость м/с, статус: active, maintenance, alarm).
- **floor_calls** — история вызовов с этажей. Фиксируем: на каком этаже нажали кнопку, в каком направлении хочет ехать пассажир (up/down), временная метка, и какой лифт в итоге был назначен (может быть null, пока не назначен).
- **trip_logs** — детальная история поездок (кабина приняла вызов -> вошёл пассажир -> нажата кнопка N -> двери закрылись -> поездка -> двери открылись). Здесь нужно хранить пройденное расстояние и время в секундах. Это нужно для расчета техобслуживания.

Основной алгоритм

Реализуйте алгоритм назначения лифта, который выбирает «лучший» лифт из доступных в здании. Просто выбрать ближайший недостаточно.

Алгоритм должен учитывать:

Приоритет статуса: Лифты в статусе alarm или maintenance исключаются.

Направление и попутность:

- Если лифт движется вверх, он может принять вызов «Вверх» на этажах выше его текущего положения.
- Если лифт движется вниз — вызов «Вниз» на этажах ниже.
- Свободный (idle) лифт может принять любой вызов.
- Минимизация времени ожидания: Оцените, через сколько секунд лифт доедет (учитывая скорость из БД, остановки и расстояние).
- Защита от перегруза: Если на этом лифте уже есть активные поездки в противоположном направлении, штрафовать такой вариант.

Методы в API

Базовые операции:

- **POST /buildings** — добавить здание.
- **POST /elevators** — добавить лифт (с валидацией: лифт не может иметь максимальный этаж больше, чем есть в здании).

Игровые механики:

- **POST /call** — Главный метод.
 - Входные данные: building_id, floor_number, direction (up/down).
 - Логика работы:
 - Записать запись в floor_calls (статус: ожидание).
 - Запустить Алгоритм поиска лифта (см. выше).
 - Если лифт найден: обновить floor_calls (назначить лифт), изменить его внутренний вектор движения, создать запись в trip_logs (начало поездки).
 - Если все лифты заняты/неисправны: вернуть ошибку 503 с сообщением о времени ожидания и поставить вызов в очередь.
 - Ответ: ID назначенного лифта, расчетное время прибытия.
- **POST /elevator/{id}/action** — Симуляция действий внутри кабины.
 - Входные данные: action_type (move_to_floor или finish_ride), target_floor.

- Логика: При завершении поездки записать в `trip_logs` итоговое расстояние и время, обновить текущее состояние лифта (текущий этаж, статус простоя).

Аналитика:

- **GET /analytics/degradation** — Поиск лифтов с риском поломки.
 - Запрос должен вернуть список лифтов, у которых суммарное пройденное расстояние за последние 30 дней превышает 100 км.
- **GET /analytics/bottlenecks** — Узкие места в здании.
 - Вернуть топ-5 этажей, где пассажиры ждут дольше всего. Нужно вычислить среднюю разницу во времени между созданием `floor_calls` и началом поездки в `trip_logs`.